

```

#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>
#include <algorithm>

using namespace std;
using namespace chrono;

//----- MAX HEAP -----//
void maxHeapify(vector<int> &arr, int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
    {
        swap(arr[i], arr[largest]);
        maxHeapify(arr, n, largest);
    }
}

void maxHeapSort(vector<int> &arr)
{
    int n = arr.size();

    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)

```

```
maxHeapify(arr, n, i);
```

```
// Heap Sort
```

```
for (int i = n - 1; i > 0; i--)
```

```
{
```

```
    swap(arr[0], arr[i]);
```

```
    maxHeapify(arr, i, 0);
```

```
}
```

```
}
```

```
//----- MIN HEAP -----//
```

```
void minHeapify(vector<int> &arr, int n, int i)
```

```
{
```

```
    int smallest = i;
```

```
    int left = 2 * i + 1;
```

```
    int right = 2 * i + 2;
```

```
    if (left < n && arr[left] < arr[smallest])
```

```
        smallest = left;
```

```
    if (right < n && arr[right] < arr[smallest])
```

```
        smallest = right;
```

```
    if (smallest != i)
```

```
    {
```

```
        swap(arr[i], arr[smallest]);
```

```
        minHeapify(arr, n, smallest);
```

```
    }
```

```
}
```

```
void minHeapSort(vector<int> &arr)
```

```
{
```

```
    int n = arr.size();
```

```
// Build Min Heap
```

```
for (int i = n / 2 - 1; i >= 0; i--)
```

```

    minHeapify(arr, n, i);

// Heap Sort
for (int i = n - 1; i > 0; i--)
{
    swap(arr[0], arr[i]);
    minHeapify(arr, i, 0);
}

// Reverse to get ascending order
reverse(arr.begin(), arr.end());
}

//----- MAIN -----//
int main()
{
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    vector<int> original(n);

    srand(time(0));

    for (int i = 0; i < n; i++)
        original[i] = rand() % 100000;

    vector<int> maxHeapArray = original;
    vector<int> minHeapArray = original;

//----- MAX HEAP SORT -----//
    auto startMax = high_resolution_clock::now();

    maxHeapSort(maxHeapArray);

```

```

auto endMax = high_resolution_clock::now();

//----- MIN HEAP SORT -----//
auto startMin = high_resolution_clock::now();

minHeapSort(minHeapArray);

auto endMin = high_resolution_clock::now();

//----- DISPLAY TIME -----//

auto nanoMax = duration_cast<nanoseconds>(endMax - startMax);
auto microMax = duration_cast<microseconds>(endMax - startMax);
auto milliMax = duration_cast<milliseconds>(endMax - startMax);
duration<double> secMax = endMax - startMax;

auto nanoMin = duration_cast<nanoseconds>(endMin - startMin);
auto microMin = duration_cast<microseconds>(endMin - startMin);
auto milliMin = duration_cast<milliseconds>(endMin - startMin);
duration<double> secMin = endMin - startMin;

cout << "\n===== MAX HEAP SORT =====\n";
cout << "Nanoseconds : " << nanoMax.count() << " ns\n";
cout << "Microseconds : " << microMax.count() << " us\n";

cout << "\n===== MIN HEAP SORT =====\n";
cout << "Nanoseconds : " << nanoMin.count() << " ns\n";
cout << "Microseconds : " << microMin.count() << " us\n";

return 0;
}

```